
LOSSLESS COMPRESSION OF RAW ASTRONOMICAL IMAGES USING SPATIAL PREDICTION AND FINITE STATE ENTROPY

Tomás Brás
University of Aveiro
112665
tomas.bras@ua.pt

David Pelicano
University of Aveiro
113391
davidpoetapelicano@ua.pt

Afonso Ferreira
University of Aveiro
113480
afonso.ferreira@ua.pt

ABSTRACT

We present three lossless compressors for raw 16-bit astronomical images, each targeting a distinct point on the speed–ratio trade-off. **speed-focused** applies a gradient-adjusted predictor (GAP) over parallel horizontal strips and codes residuals with a context-adaptive range coder, reaching 3.505 bpp in 49 ms total. **all-rounder** selects the best of five per-block predictors (including training-free least-squares with up to five spatial taps) and codes residuals with Finite State Entropy (tANS), achieving 3.400 bpp in 96 ms. **compression-focused** exhausts 60 predictor-context combinations and seven block sizes per image, using least-squares predictors with up to 24 spatial taps and an adaptive range coder, reaching 3.367 bpp. All three codecs outperform every tested general-purpose compressor in compression ratio, with all-rounder and speed-focused also beating bzip2-9 and xz-6 in total runtime.

1 Introduction

Raw images produced by ground-based astronomical telescopes are large, structured numerical arrays. Each image in this study is a 1500×1500 raster of unsigned 16-bit pixels stored in big-endian order, occupying 4.5 MB. These arrays differ from conventional files in three important ways. First, adjacent pixels are spatially correlated: the sky background and optical point-spread function create smooth gradients that extend over many pixels. Second, the readout electronics add a constant *bias pedestal* to every pixel, introducing a strong DC component whose removal collapses the residual distribution around zero. Third, the data are 16-bit integers, whereas most general-purpose compressors operate on byte streams, missing inter-byte structure.

These properties motivate *predictive* compression: subtract a spatial prediction $\hat{p}$ from each pixel, code only the residual $r = p - \hat{p}$, and exploit the fact that a good predictor concentrates the residuals near zero, thereby reducing entropy and improving compression.

Existing domain-specific tools such as the FITS FPACK suite (Rice coding, Hcompress) operate after splitting pixels into individual bytes, thereby discarding inter-byte correlation. Our codecs handle 16-bit values natively and select the best predictor per block rather than applying a fixed rule to the whole image. This differs also from recent learned-predictor approaches such as Quintas-Torra et al. [5], which train a weight matrix on a held-out set from the same sensor: our least-squares predictors solve per-block OLS on the image itself with no external training, making them immediately applicable to any sensor or telescope without retraining.

We present three lossless compressors that exploit these properties, each targeting a distinct point on the speed–ratio trade-off:

- **speed-focused** applies a gradient-adjusted predictor (GAP) to the full 16-bit value and codes residuals with a range coder driven by eight context-conditioned adaptive models across parallel horizontal bands. It prioritises throughput (~ 49 ms per image).
- **all-rounder** divides the image into square blocks, selects the best of five candidate predictors per block (including training-free least-squares), splits residuals into independent byte streams, and codes them with Finite State Entropy (FSE/tANS) [2]. It balances speed and ratio (~ 96 ms, 3.40 bpp).
- **compression-focused** evaluates 60 combinations of ten predictors (including LS-16 with 16 spatial neighbours and LS-24 with 24 neighbours) and six context strategies per block, selects the smallest, and uses an LZMA-style adaptive range coder with Fenwick-tree models. Block size is also optimised by trying seven candidates at compression time. It prioritises compression ratio (~ 3.37 bpp) at the cost of a slower compression pass (~ 6.2 s); decompression is fast (~ 133 ms).

The remainder of this paper is organised as follows. Section 2 describes the three codecs in detail, Section 3 defines the experimental protocol, Section 4 reports compression ratios and runtimes, and Section 5 draws conclusions.

2 Codec Design

All three codecs share the same high-level pipeline: parse the raw big-endian 16-bit image; apply a spatial predictor $\hat{p}$; zigzag-encode the signed residual $r = \text{pixel} - \hat{p}$ as an unsigned symbol $z = 2r$ if $r \geq 0$, $z = -2r - 1$ otherwise; entropy-code the result. Zigzag maps small-magnitude residuals (the common case after a good prediction) to small unsigned integers, concentrating the symbol distribution near zero regardless of sign.

2.1 *speed-focused*

The design of *speed-focused* centres on one principle: every horizontal strip of 150 rows is entirely self-contained, so any number of strips can be compressed simultaneously without coordination. A shared `std::atomic<int>` counter distributes work to a thread pool via `fetch_add`, achieving lock-free load balancing with no mutex overhead.

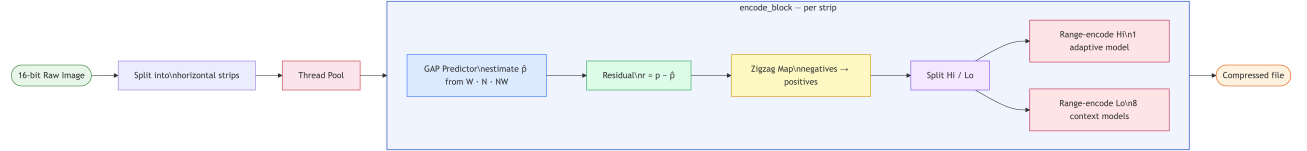


Figure 1: *speed-focused* pipeline. Each strip is processed independently by a worker thread: GAP prediction, zigzag mapping, hi/lo byte split, and adaptive range coding with eight context models.

Within each strip, pixels are predicted using the *Gradient-Adjusted Prediction* (GAP) function of three causal neighbours: left (W), above (N), and above-left (NW). GAP estimates horizontal and vertical gradients as $d_h = |W - NW|$ and $d_v = |N - NW|$, and when one gradient strongly dominates ($d_h > 2d_v$ or $d_v > 2d_h$) it falls back to the perpendicular neighbour directly. Otherwise it computes a gradient-weighted blend clamped to the range $[\min(W, N), \max(W, N)]$:

$$\hat{p} = \text{clamp}\left(\frac{(d_v + 1)W + (d_h + 1)N}{d_v + d_h + 2}, \min(W, N), \max(W, N)\right).$$

This single formula handles both smooth regions and edges without any explicit mode switch.

The signed residual is zigzag-mapped to a 16-bit unsigned symbol and split into a high byte $z_h = z \gg 8$ and a low byte $z_l = z \& 0xFF$. Both are coded with a carry-propagation range coder backed by adaptive Fenwick-tree frequency models. The high byte uses one shared model across all pixels in the strip, while the low byte selects among eight context models conditioned on the sum of the two neighbouring residual magnitudes:

$$c = \text{bin}(|\hat{r}_W| + |\hat{r}_N|),$$

with bin boundaries $\{0\}, \{1-2\}, \{3-8\}, \{9-32\}, \{33-64\}, \{65-128\}, \{129-512\}, \{> 512\}$. Each model halves all counts when the running total reaches 2^{16} , keeping the frequency estimates responsive to distribution shifts within a strip.

2.2 *all-rounder*

Block Selection

all-rounder requires explicit `n_rows` and `n_cols` arguments, which are stored in the compressed header so the decoder needs no additional input. It then selects the largest block side $b \leq 512$ that divides both W and H exactly and yields at least four blocks. For 1500×1500 this gives $b = 500$, producing a 3×3 grid of nine blocks of 500×500 pixels.

Predictive Modes

For each block, *all-rounder* evaluates five candidate predictors and selects the one with the lowest estimated coding cost. **Mode 0 (raw)** applies no prediction and codes pixel values directly; it wins only on blocks where spatial correlation has been destroyed by strong read-noise. **Mode 1 (avg)** predicts each pixel as the three-neighbour average $\hat{p} = \lfloor (W + N + NW + 1)/3 \rfloor$, which smooths slowly varying stellar backgrounds effectively. **Mode 3 (gmean)** subtracts the global image mean $\bar{g}$ (computed once and stored in the 2-byte file header) from every pixel; this single operation removes the bias pedestal that dominates calibration frames, collapsing the residual distribution to near-zero at no per-block cost. **Mode 4 (LS-4)** solves a per-block ordinary least-squares regression with features $(W, N, NW, 1)$ via Gauss-Jordan elimination, producing four float32 weights (16 B block overhead) that jointly adapt to local spatial gradients and residual DC offset. **Mode 6 (LS-5)** extends Mode 4 with above-right (NE) and two-rows-above (NN) neighbours, adding a fifth weight (20 B total) to capture vertical second-order structure present in science frames with extended emission.

The mode selection criterion penalises stored weights:

$$\text{cost}_m = H(\text{hi}_8) + H(\text{lo}_8) + \frac{8\delta_m}{n_{\text{pix}}},$$

where $H(\cdot)$ is empirical byte entropy and $\delta_m \in \{0, 0, 0, 16, 20\}$ B is the weight header size.

Entropy Coding: FSE/tANS

Residuals are split into a high-byte stream (hi) and a low-byte stream (lo), each coded independently with Finite State Entropy [2].

The FSE table has $SCALE = 2^{16} = 65536$ states. Raw symbol counts are normalised to sum to $SCALE$ (with a minimum frequency of 1 for observed symbols) and spread deterministically using Collet’s step formula: $step = (SCALE \ggg 1) + (SCALE \ggg 3) + 3$. The encoder processes symbols in reverse, emitting variable-length transition bits stored in a `BitWriter`; the decoder reads the two-byte final state and reconstructs symbols in forward order.

The frequency table is serialised in one of three compact formats: *single-symbol* (flag $0x01$, 2 B total) when all values are identical; *sparse* (flag byte = $nnz \in [2, 254]$, followed by $nnz \times 5$ B symbol–frequency pairs); or *dense* (flag $0x00$, 256×4 B) when all 255–256 symbols appear.

Context FSE

Because the high byte of a small residual is almost always zero, the $lo8$ distribution differs substantially depending on whether $hi = 0$. When the fraction of zero high bytes falls between 2% and 98%, ***all-rounder*** builds two conditional $lo8$ streams: lo_zero (pixels with $hi = 0$) and $lo_nonzero$, and uses the three-stream layout only if its compressed size is strictly smaller than the two-stream baseline. The high-byte stream is encoded once and shared between both candidate payloads. Bit 7 of the mode byte signals the active layout.

Parallel Execution

Blocks are dispatched to a thread pool via a single `std::atomic<int>` counter with `fetch_add`, writing results into pre-allocated per-block slots with no mutex required.

2.3 compression-focused

compression-focused separates orchestration from compression. The outer `compress` binary runs the inner `range_compress` engine seven times with block sizes $\{250, 300, 400, 500, 750, 1000, 1500\}$ pixels in parallel, retains the smallest result, and wraps it with a 5-byte magic header. The corresponding `decompress` / `range_decompress` pair inverts the process. This exhaustive search over block sizes incurs extra compression time but requires no tuning per image.

Figure 2 summarises the full pipeline.

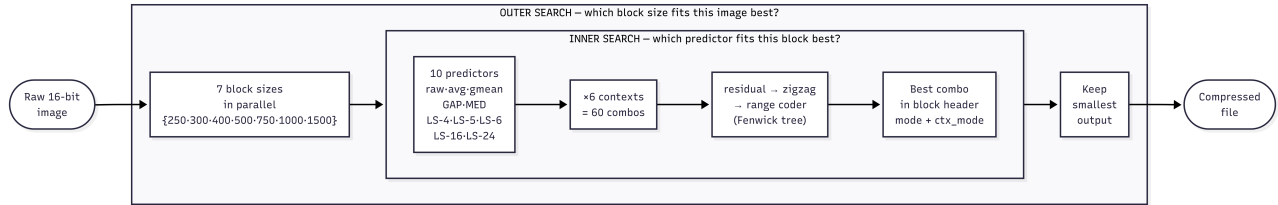


Figure 2: ***compression-focused*** pipeline. The outer search runs `range_compress` seven times in parallel (one per block size) and retains the smallest output. Inside each run, every block is compressed under all $10 \times 6 = 60$ predictor/context combinations and the best is stored in the block header.

Predictor Competition

compression-focused evaluates ten predictors per block, spanning the full range from non-predictive to highly adaptive. Modes 0, 1, 3, GAP, and MED cover the same ground as ***all-rounder***’s five modes (raw, three-neighbour average, global mean, gradient-adjusted, and JPEG-LS median), providing a strong baseline across all image types. The five OLS predictors extend the context progressively: LS-4 uses $(W, N, NW, 1)$ with 4 weights (16 B); LS-5 adds NE and NN for 5 weights (20 B); LS-6 adds second-order neighbours WW and NN for 6 weights (24 B); LS-16 expands to a 16-tap context spanning four pixels left and seven pixels above (64 B); and LS-24 uses a 24-tap context covering three full rows around the current pixel (96 B). All LS weights are solved per block via Gauss-Jordan elimination with no external training. Border pixels where the full neighbourhood is unavailable fall back to GAP. The winner is chosen by actual compressed size across all $10 \times 6 = 60$ predictor-context combinations, eliminating estimation error.

Context Modes and Block Competition

Each 16-bit zigzag residual is encoded byte-by-byte: first the high byte (z_h), then the low byte (z_l). Six context strategies are tried, combining hi -byte and lo -byte conditioning:

- **ctx 0**: single hi model; single lo model.
- **ctx 1**: single hi model; four lo models keyed by $z_h \in \{0\}, \{1-2\}, \{3-8\}, \{> 8\}$.
- **ctx 2**: single hi model; 256 lo models (one per z_h value).

- **ctx 3–5**: same lo conditioning as ctx 0–2, but the hi byte is coded with *four* models conditioned on the previous hi byte $z_h^{(t-1)}$ using the same four-bin scheme. This order-1 hi context exploits spatial runs of similarly sized residuals.

Every block is compressed under all $10 \times 6 = 60$ mode/context combinations; the combination producing the smallest byte count is stored. Each block header records the chosen mode (1 B) and ctx_mode (1 B) so the decoder can reproduce the exact prediction and model layout without side information.

Entropy Coding: Adaptive Range Coder

compression-focused uses an LZMA-style range coder with a 32-bit range register and a 64-bit low accumulator. Carry propagation is handled by a cache/pending chain: bytes whose top nibble is 0xFF are held until a carry either flushes them as 0x00 or confirms them as 0xFF. Renormalisation triggers when $\text{range} < 2^{24}$, emitting one output byte and shifting the range left by 8.

The probability model is a 256-symbol adaptive Fenwick tree. Frequencies are initialised uniformly at 1. After each symbol, the count is incremented in $O(\log 256)$ time. When the running total reaches 2^{16} , all frequencies are halved (minimum 1) and the tree is rebuilt, implementing a sliding-window aging mechanism that keeps the model responsive to local distribution shifts.

3 Experimental Setup

3.1 Dataset

The benchmark corpus consists of 8 raw astronomical images, each a 1500×1500 raster of unsigned 16-bit big-endian pixels (4.5 MB per file, 2.25×10^6 pixels). The images were acquired by different ground-based telescopes (VLT, TJO, GTC, INT, JKT, LCO, WHT) and span a deliberate variety of image types: bias frames (near-constant DC pedestal), flat fields (smooth illumination gradients), and science frames (stellar fields, galaxies). This diversity ensures that no single predictor type dominates the results artificially.

Table 1 characterises each image by its zero-order entropy H_0 (bits per pixel, treating pixels as i.i.d.) and mean pixel value $\bar{p}$. These quantities explain predictor choice: low-entropy bias and flat frames are dominated by a DC pedestal that Mode 3 (global mean subtraction) removes in a single step; high-entropy science frames retain spatial structure after mean removal, making data-adaptive LS predictors beneficial.

Table 1: Benchmark image characterisation.

Image	Type	$\bar{p}$	H_0 (bpp)
A	Stellar field	16199	10.27
B	Mixed sky	2396	6.81
C	Dark frame	610	4.87
D	Bias frame	1038	5.43
E	Flat field	113	5.11
F	Science (sources)	30456	12.52
G	Bias frame	4073	5.57
H	Calibration	1550	6.61

3.2 External Generalisation Dataset

To assess generalisation, we also tested *all-rounder* on the dataset of Quintas-Torra et al. [5], consisting of 225 raw astronomical images from five telescopes (INT, JKT, LCO, TJO, WHT) with dimensions ranging from 510×765 to 4200×2154 pixels and sensor types different from those in the benchmark corpus. No codec parameters were retuned for this dataset.

3.3 Reference Compressors

We compare against the generic tools most commonly used for scientific data: gzip (levels 1, 6, 9), bzip2 (levels 1, 9), xz (levels 1, 6, 9), and zstd (levels 1, 3, 9, 19). All are invoked on the raw byte stream without any pre-processing or endian conversion.

3.4 Evaluation Protocol

The primary metric is bits per byte of the original file:

$$\text{bpp} = \frac{8 \times \text{compressed bytes}}{\text{original bytes}}.$$

For a 16-bit image, $\text{bpp} = \text{bits per pixel}/2$. Compression and decompression times are measured as wall-clock elapsed time for a single run on a single machine; all compressors run sequentially in the same benchmark process to avoid

contention. Losslessness is verified by byte-exact comparison of the decompressed output against the original file; any mismatch disqualifies the result. Binary size constraints from the project specification are satisfied: compress ≤ 5 MB, decompress ≤ 1 MB.

4 Results

4.1 Compression Ratio

Table 2 reports bits per byte for each of the eight benchmark images. All three domain-specific codecs outperform every generic compressor on every image. *compression-focused* achieves the best average ratio (3.37 bpp), reducing average bpp by 0.21 versus bzip2-9 and by 0.81 versus zstd-9. *all-rounder* follows closely (+0.03 bpp over *compression-focused*) and *speed-focused* is 0.11 bpp above *all-rounder*, still beating every generic tool.

The margin is consistent across image types. Image F (science frame with bright point sources, $H_0 = 12.52$ bpp) is the hardest for all codecs because its high raw entropy leaves little room for prediction; even so, *compression-focused* saves 0.41 bpp over bzip2-9 on that image alone. Image G (bias frame, $H_0 = 5.57$ bpp) is the easiest: the near-constant DC pedestal is entirely captured by Mode 3 and compressed to 2.38 bpp, 0.16 bpp below bzip2-9.

Table 2: Compression ratio (bpp) per image

Image	Our codecs			Generic	
	speed	all	compr.	bzip2-9	zstd-9
A	4.44	4.39	4.35	4.62	5.51
B	3.29	3.24	3.21	3.40	4.07
C	2.59	2.43	2.41	2.59	3.07
D	2.85	2.73	2.71	2.89	3.45
E	2.73	2.56	2.56	2.71	3.22
F	6.20	6.14	6.06	6.47	7.00
G	2.55	2.41	2.38	2.54	3.03
H	3.41	3.29	3.25	3.41	4.11
Mean	3.51	3.40	3.37	3.58	4.18

4.2 Speed

Table 3 shows average per-image runtimes. *speed-focused* achieves 49 ms total, only 22 ms slower than zstd-1 but with 0.98 bpp better compression. *all-rounder* takes 96 ms, $4.5\times$ faster than bzip2-9 (432 ms) at 0.18 bpp better ratio. *compression-focused* trades a slow compression pass (~ 6.2 s) for the best ratio; decompression (~ 133 ms) is faster than bzip2-9.

Table 3: Average per-image compressed size, runtimes (ms), and bpp. Original file: 4.5 MB.

Compressor	Size (MB)	Compress (ms)	Decompress (ms)	Total (ms)	bpp
<i>speed-focused</i>	1.97	23	26	49	3.51
<i>all-rounder</i>	1.91	56	40	96	3.40
<i>compression-focused</i>	1.89	6169	133	6302	3.37
bzip2-9	2.01	273	159	432	3.58
xz-6	2.07	1650	102	1752	3.68
zstd-9	2.35	128	11	139	4.18
zstd-1	2.52	20	7	27	4.49

5 Discussion and Conclusion

Generic compressors treat the two bytes of each 16-bit pixel as unrelated, breaking the spatial correlation that spans pixel boundaries in big-endian encoding. Our codecs operate on 16-bit values directly, predict each pixel from its decoded neighbours, and entropy-code only the residual, yielding a reduction of 0.87–1.01 bpp over gzip-6 depending on the codec.

The predictor turns out to be the dominant design choice. Table 4 shows the per-block mode selection produced by *all-rounder* on each benchmark image. Mode 3 (gmean) dominates on four of eight images: it is the sole predictor for images D and H, which are bias frames with a near-constant DC pedestal, and the majority predictor for E and G. Images A and B, which have smooth or mixed stellar backgrounds, are handled by Mode 1 (avg), which better tracks slowly varying spatial gradients. Image F is the exception — a science frame containing bright point sources and extended emission where LS-4 and LS-5 account for all nine blocks, yielding the largest per-image gain over generics (0.33 bpp over bzip2-9 for *all-rounder*). Image C uses Mode 0 (raw) on six blocks, suggesting that dark calibration frames with dominant Poisson noise lose spatial correlation at block scale. Quantitatively, prediction accounts for 0.67 bpp of the 0.78 bpp gain over zstd-9; the remaining 0.11 bpp comes from the choice of entropy coder, confirming that the predictor is the binding component.

Table 4: *all-rounder* mode selection per image (blocks out of 9).

Image	raw	avg	gmean	LS-4	LS-5
A	0	9	0	0	0
B	0	6	3	0	0
C	6	0	3	0	0
D	0	0	9	0	0
E	3	0	6	0	0
F	0	0	0	4	5
G	0	2	6	0	1
H	0	0	9	0	0

On the entropy coding side, splitting residuals into independent hi8 and lo8 streams is the key structural choice. After a good prediction, hi8 is near-uniformly zero and is often compressed to a 2-byte FSE packet, leaving all meaningful variation to the lo8 stream. *speed-focused* achieves the same effect through eight adaptive context models without transmitting a frequency table, trading 0.11 bpp for a roughly $2\times$ speed advantage over *all-rounder* (49 vs 96 ms). *compression-focused*'s exhaustive 10×6 search recovers only 0.03 bpp over *all-rounder*, which further confirms that predictor quality is the binding constraint rather than the entropy coder.

To assess generalisation, we tested *all-rounder* on the dataset from Quintas-Torra et al. [5] (INT, JKT, LCO, TJO, and WHT telescopes; 225 images with dimensions ranging from 510×765 to 4200×2154). Mean bpp per telescope was INT 2.71, JKT 2.85, TJO 2.78, WHT 3.07, and LCO 3.36. LCO is the hardest dataset, mixing full-frame science images with small calibration frames whose shorter spatial runs give the predictor less context, pushing bpp up to 6.88 in the worst case. WHT also compresses less efficiently due to high raw entropy driven by dense stellar fields and strong read-noise, while INT and TJO benefit from large uniform sky backgrounds and stable bias pedestals. These results confirm that the codecs generalise across sensor geometries and telescope types without any retraining.

Overall, all three codecs outperform every tested generic compressor in compression ratio, and two of the three also beat bzip2-9 in total runtime. The consistent margin across all eight benchmark images, together with the generalisation to 225 external images from five telescopes, confirms that the gains are structural rather than dataset-specific. Both encoder and decoder accept explicit row and column dimensions, making the codecs applicable to any sensor geometry.

References

- [1] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [2] Yann Collet. Finite State Entropy: a new entropy coder. <https://github.com/Cyan4973/FiniteStateEntropy>, 2013.
- [3] Yann Collet. Zstandard: fast real-time compression algorithm. <https://github.com/facebook/zstd>, 2016.
- [4] William D. Pence, L. Seaman, R. Seaman, and R. White. FITS lossless image compression. *Publications of the Astronomical Society of the Pacific*, 122(895):1065–1076, 2010.
- [5] Pau Quintas-Torra, Xavier Fernández-Mellado, Joan Bartrina-Rapesta, Sebastià Mijares, Armando J. Pinho, and Joan Serra-Sagristà. Lossless compression of modern astronomical data using a novel learned predictor. *Publications of the Astronomical Society of the Pacific*, 138:044505, 2026.
- [6] Michele Trovato, Claudio Talarico, Rosario Catanzaro, and Antonino Tumeo. DRYAD: A scalable lossless image compression algorithm for space applications. In *Proc. Data Compression Conference (DCC)*, 2018.